Sistema de Integración y Análisis de Ventas

Documento explicativo - Programación II - 30 de octubre de 2026
Jonathan Daniel Alberto Aguilar Ramírez y Luis Alberto Matta Rubio

1. Propósito y arquitectura

La aplicación de consola reúne ventas de archivos CSV, JSON y XML. Las convierte a objetos, normaliza textos, rechaza datos inválidos, detecta IDs repetidos y conserva las ventas válidas en PostgreSQL. Después consulta los datos consolidados y genera dos archivos de texto. Se mantuvo la arquitectura del proyecto existente, sin reconstruirlo ni incorporar una interfaz gráfica.

Main coordina el proceso. ProcesadorVentas descubre archivos y mantiene los contadores. Los lectores interpretan cada formato y delegan la conversión común a ConversorVenta. RegistroVenta transporta los ocho campos y el origen. ValidadorVenta contiene las reglas y DeduplicadorVentas compara identificadores.

ConexionBD obtiene la configuración privada; VentaDAO consulta e inserta mediante JDBC. AnalizadorVentas reúne las consultas SQL para que consola y reporte usen la misma lógica. GeneradorReportes combina estadísticas y consultas y escribe las salidas. El punto de entrada antiguo delega en Main; las clases Producto y Venta permanecen como antecedente del prototipo.

2. Contrato de entrada

Los campos son id_venta, fecha, sucursal, codigo_producto, producto, cantidad, precio_unitario y vendedor. Se utiliza UTF-8. CSV requiere encabezados, permite otro orden y admite comillas, comas internas, BOM y campos multilínea mediante Apache Commons CSV. JSON utiliza un arreglo de objetos con Jackson. XML requiere ventas como raíz y venta como hijo directo; se bloquean DTD y entidades externas.

Se aceptan diferencias de mayúsculas en encabezados y etiquetas, pero no se inventan alias ni envoltorios ausentes en las entradas originales. Las estructuras admitidas están documentadas en el README. Un archivo físicamente vacío o mal formado se registra como fallo de archivo. Una colección vacía bien formada contiene cero ventas.

3. Recorrido verificable

La venta V001 del CSV se convierte en un objeto con fecha LocalDate, cantidad entera y precio BigDecimal. Su importe es cantidad por precio. El JSON incluye v001: después de normalizarlo se reconoce como duplicado. Se conserva el primer registro válido, se registra el segundo y se mantienen ambos orígenes en el mensaje. Una venta inválida no llega al DAO.

Los archivos nuevos que respetan el contrato se colocan en entrada. Se descubren en orden de nombre, sin modificar Main. También pueden indicarse rutas explícitas y un directorio de salida.

Validaciones, excepciones y persistencia

4. Reglas compartidas

Todos los campos son obligatorios. La fecha debe ser real y estar entre los años 0001 y 9999. La cantidad debe ser un entero de 32 bits mayor que cero. El precio debe ser positivo y compatible con NUMERIC(12,2): máximo 9999999999.99 y dos decimales significativos. Se rechazan valores incompatibles en lugar de redondearlos silenciosamente. Se respetan las longitudes VARCHAR del esquema.

La normalización recorta y colapsa espacios Unicode. ID, sucursal y código pasan a mayúsculas con Locale.ROOT. Producto y vendedor conservan su presentación; SQL agrupa sin distinguir mayúsculas. No se cambian cantidades, precios, fechas ni nombres para hacer pasar un registro inválido.

5. Excepciones y continuidad

Las conversiones numéricas y de fecha tienen tratamiento específico. Un registro puede acumular varios motivos en una incidencia, pero cuenta como un solo rechazo. Cada mensaje identifica archivo, registro e ID cuando está disponible. Un error de registro no impide procesar el siguiente.

Los errores estructurales JSON, XML o CSV impiden aceptar fragmentos ambiguos del archivo. Se continúa con los demás archivos. Se distinguen errores de lectura, conversión, validación y JDBC. Los archivos, conexiones, sentencias y resultados se cierran mediante try-with-resources. Los mensajes JDBC incluyen SQLState y no publican contraseñas.

6. Duplicados y PostgreSQL

DeduplicadorVentas compara el ID normalizado y conserva el primero válido. Distingue repeticiones idénticas de IDs con datos contradictorios. Los precios se comparan por valor económico, sin confundir 250 con 250.00.

VentaDAO usa PreparedStatement con parámetros, clave primaria y ON CONFLICT DO NOTHING. Una ejecución repetida encuentra la fila existente sin insertarla otra vez. Si el contenido difiere, se conserva lo almacenado y se registra conflicto. El esquema nuevo impide campos vacíos, ID no canónico y cantidades o precios no positivos. IF NOT EXISTS no migra una tabla antigua: cualquier cambio sobre ella necesita revisión.

7. Seguridad y reproducibilidad

La configuración local se obtiene de un archivo ignorado o variables de entorno. Durante la auditoría se detectó que el archivo privado estaba versionado. Se retiró del índice conservando su copia local. Los commits anteriores no se reescribieron; corresponde revisar exposición y rotar credenciales si fueron publicadas.

Las pruebas de integración usan pg_temp.ventas, verifican que la tabla sea temporal y ejecutan ROLLBACK. Además comparan una huella de public.ventas antes y después. Los reportes personales se regeneraron en modo de solo lectura. No se insertaron, eliminaron ni sobrescribieron ventas personales.

Indicadores, evidencias y pendientes

8. Contadores y análisis SQL

Los contadores de la ejecución cumplen tres igualdades: recibidos = válidos + rechazados; válidos = únicos + duplicados; únicos = insertados + existentes + conflictos + fallos JDBC. El porcentaje es rechazados / recibidos por 100, o cero si no hay registros. Los archivos ilegibles no aportan una cantidad inventada de registros.

Los indicadores monetarios se consultan sobre toda la información almacenada, separados de los contadores de la ejecución. SQL calcula COUNT, SUM y AVG del importe por venta. GROUP BY obtiene ventas por mes, sucursal, producto y vendedor. ORDER BY y LIMIT permiten extremos y top 3; las etiquetas desempatan. El análisis usa una instantánea consistente de solo lectura.

9. Hallazgos comprobados

La versión inicial compilaba, pero carecía de JUnit. El CSV contaba su encabezado como venta porque el archivo comenzaba con una línea vacía; tampoco admitía comas entrecomilladas. El script incluía instrucciones de MySQL y Main solo leía tres nombres fijos. Los fallos de archivo se mezclaban con registros rechazados y las consultas estaban duplicadas entre consola y reporte.

Con los archivos actuales se verificaron 11 registros recibidos, 5 válidos antes de deduplicar, 6 rechazados, 1 duplicado y 4 únicos. La tasa de rechazo correcta es 54,55 %, no el 58,33 % anterior. Las cuatro ventas acumuladas suman Q11350,00; el promedio por transacción es Q2837,50. Carlos reúne Q9500,00 y la transacción V007 alcanza Q9000,00. Estos son resultados medidos, no conclusiones personales.

10. Pruebas y alcance

Pasaron 56 pruebas: 52 casos de procesamiento y 4 de integración PostgreSQL, sin errores, fallos ni omisiones en la ejecución completa. Se verificaron campos obligatorios, fechas imposibles, cantidades y precios inválidos, normalización, comillas CSV, documentos mal formados, archivos vacíos, duplicados entre formatos, reejecución y correspondencia del reporte con SQL.

La compilación limpia, las pruebas y el JAR se ejecutaron con un JDK 17 real. También se verificó Java 26 con destino 17. Las instrucciones de NetBeans están documentadas; no se realizó una comprobación interactiva de su interfaz. Los casos privados del docente no están disponibles.

11. Intervención de los integrantes

Jonathan debe redactar tres conclusiones propias y Luis otras tres, vinculadas a resultados y limitaciones reales. Los seis apartados están identificados en el reporte; no se inventó su interpretación. También deben revisar credenciales históricas y evidenciar su participación real en Git.

Se localizó el enunciado original en PROYECTO DE CURSO.html. Sus secciones difieren en los puntajes de proyecto y defensa; el equipo debe consultar esa inconsistencia con el docente. La guía de defensa ayuda a seguir un registro, modificar una regla y depurar un error. La entrega académica no está completa mientras falten conclusiones y esas revisiones humanas.